

DataFrameLib

Design Document

COP290 — Design Practices in Programming

IIT Delhi

Contents

1	Overview	2
2	Architecture	2
2.1	Components	2
2.2	Apache Arrow as the Storage Layer	2
2.3	Type Safety	2
2.4	Memory Management	2
3	Expression System	3
3.1	Node Types	3
3.2	Evaluation	3
3.3	Factory Interface	3
4	Eager Execution	3
5	Lazy Execution	4
5.1	Plan Construction	4
5.2	Collect	4
5.3	DAG Rendering	4
6	Query Optimiser	4
6.1	Predicate Pushdown	4
6.2	Limit Pushdown	5
6.3	Projection Pushdown	6
6.4	Constant Folding	6
6.5	Expression Simplification	7
7	Design Decisions	8

1 Overview

DataFrameLib is a C++ library for tabular data processing. It exposes two execution modes — **eager** and **lazy** — backed by Apache Arrow for all in-memory storage and I/O. A rule-based query optimiser transforms lazy execution plans before materialisation, reducing both memory usage and computation time.

2 Architecture

2.1 Components

The library is organised into four modules:

Module	Files	Responsibility
Expression system	<code>expr.h</code> / <code>expr.cpp</code>	Expression tree nodes and evaluation
Eager execution	<code>dataframe.h</code> / <code>dataframe.cpp</code>	Immediate, materialised operations
Lazy execution	<code>lazy.h</code> / <code>lazy.cpp</code>	Deferred DAG construction and optimiser
I/O utilities	<code>utils.cpp</code>	CSV/Parquet reading and writing

2.2 Apache Arrow as the Storage Layer

All data is held in `arrow::Table` objects — immutable, columnar, reference-counted structures. This gives:

- Cache-friendly column scans.
- Zero-copy sharing between operations via `shared_ptr`.
- Built-in null representation (Arrow null bitmap, not NaN).
- Direct CSV and Parquet I/O without intermediate conversion.

2.3 Type Safety

Each column has an immutable Arrow type (`int32`, `int64`, `float32`, `float64`, `string`, `boolean`). Mixed-type arithmetic follows the promotion rule $int + float = float$. Any operation involving a null operand produces a null result. Incompatible operations (e.g. adding a numeric and a string column) throw a `std::runtime_error` immediately.

2.4 Memory Management

All Arrow objects are wrapped in `std::shared_ptr`. `EagerDataFrame` holds a single `shared_ptr<arrow::Table>`; operations produce new frames without mutating existing ones. There is no manual `new/delete`.

3 Expression System

3.1 Node Types

The expression tree consists of four node types, all inheriting from `Expr`:

Node	Description
<code>LiteralExpr</code>	A constant scalar value
<code>ColumnExpr</code>	A reference to a named column
<code>UnaryExpr</code>	One operand: <code>NOT</code> , <code>ABS</code> , string ops, null checks
<code>BinaryExpr</code>	Two operands: arithmetic, comparison, boolean
<code>AggExpr</code>	Aggregation over a column: <code>SUM</code> , <code>MEAN</code> , etc.

3.2 Evaluation

Each node implements `evaluate(table)` returning a `vector<shared_ptr<arrow::Scalar>>` of length `num_rows`. Null propagation is handled at the scalar level: any arithmetic or comparison involving a null scalar returns a null scalar.

3.3 Factory Interface

The public API uses free functions and operator overloads so that expressions read naturally:

```
(col("age") > 30) & (col("dept") == "Engineering")
col("salary") * 0.1 + lit(500)
~col("x").is_null()
col("name").upper().contains("ALICE")
```

4 Eager Execution

`EagerDataFrame` wraps a single `arrow::Table` and provides immediate, materialised operations. Each method returns a new `EagerDataFrame`; the original is never modified.

Method	Description
<code>select(cols)</code>	Retain named columns
<code>filter(predicate)</code>	Retain rows satisfying the predicate
<code>with_column(name, expr)</code>	Add or replace a column
<code>sort(cols, asc)</code>	Sort rows
<code>sort_head(cols, asc, n)</code>	Partial sort, return top n
<code>head(n)</code>	First n rows
<code>group_by(keys)</code>	Returns a <code>GroupedDataFrame</code>
<code>aggregate(agg_map)</code>	Aggregate a grouped frame
<code>join(other, keys, how)</code>	Inner, left, or outer join
<code>write_csv / write_parquet</code>	Persist to disk

5 Lazy Execution

5.1 Plan Construction

`LazyDataFrame` stores a source path and a `vector<Operation>`. Each chained call (e.g. `.filter(...)`, `.select(...)`) appends an `Operation` to this vector and returns a copy of the frame — no data is read or processed.

5.2 Collect

Calling `collect()` triggers:

1. The optimiser transforms `ops_` (see Section 6).
2. The source file is read into an `EagerDataFrame`.
3. Optimised operations are applied in order.

`sink_csv` and `sink_parquet` call `collect()` and immediately write the result.

5.3 DAG Rendering

`explain(path)` writes a Graphviz `.dot` file representing the optimised plan. Each node is labelled with its operation type. If Graphviz is installed, a `.png` is also rendered. This is purely for introspection and does not affect execution.

6 Query Optimiser

The optimiser applies rule-based transformations to the operation list before execution. Each rule is described below with a correctness proof.

6.1 Predicate Pushdown

Transformation

`FILTER` operations are moved as early as possible in the plan, just after the nearest preceding operation that affects row identity. They are never pushed past `GROUP_BY`, `AGGREGATE`, `JOIN`, `SORT`, or `HEAD`.

Correctness

Theorem 1. *For a row-independent operation op that does not introduce any column referenced by predicate p ,*

$$filter(p, op(T)) = op(filter(p, T)).$$

Proof. `filter(p, T)` selects the subset $S \subseteq T$ of rows satisfying p . A row-independent operation op maps each row independently (e.g. `WITH_COLUMN` appends a derived value, `SELECT` drops columns). Neither changes which rows exist nor reorders them. Since p does not reference any column introduced by op , evaluating p before or after op produces the same truth value for every row. Therefore the two orderings yield identical row sets. $\square$

The push is blocked at **GROUP_BY** (rows are merged), **AGGREGATE** (row count changes), **JOIN** (new rows may appear or disappear), **SORT** and **HEAD** (pushing past these could change which rows survive the limit).

Example

```
Before:  SCAN -> WITH_COLUMN("bonus", salary*0.1) -> FILTER(age > 30)
After:   SCAN -> FILTER(age > 30) -> WITH_COLUMN("bonus", salary *0.1)
```

Performance benefit

Filtering early reduces the number of rows that all downstream operations must process, shrinking both memory usage and CPU time proportionally to the selectivity of the predicate.

6.2 Limit Pushdown

Transformation

HEAD(n) is moved as early as possible, before **WITH_COLUMN**, **FILTER**, and **SELECT** nodes, stopping at **SORT**, **GROUP_BY**, **AGGREGATE**, and **JOIN**. A **HEAD** immediately following a **SORT** is fused into a single **SORT_HEAD**(n) call.

Correctness

Theorem 2. *For any row-independent, order-preserving operation op ,*

$$head(n, op(T)) = op(head(n, T)).$$

Proof. **head**(n, T) returns the first n rows of T in their current order. A row-independent, order-preserving op maps each row independently without reordering. Therefore the first n rows of $op(T)$ are exactly op applied to the first n rows of T . $\square$

The push is blocked at **SORT** (the limit depends on sort order), **AGGREGATE** (output row count is determined by the aggregation), and **JOIN** (which can duplicate or remove rows unpredictably).

For the fusion: **sort_head**($cols, n$) is defined to return the same result as **sort**($cols$).**head**(n) by specification, so correctness is immediate.

Example

```
Before:  SCAN -> SELECT(name, salary) -> HEAD(10)
After:   SCAN -> HEAD(10) -> SELECT(name, salary)

Before:  SCAN -> SORT(salary, desc) -> HEAD(5)
After:   SCAN -> SORT_HEAD(salary, desc, n=5)      [fused]
```

Performance benefit

An early limit avoids materialising the full dataset into downstream operations. The `sort_head` fusion replaces an $O(N \log N)$ full sort with an $O(N \log k)$ partial sort (heap-select), which is substantially faster for small k .

6.3 Projection Pushdown

Transformation

`SELECT` operations are moved as early as possible, before `FILTER` and other `SELECT` nodes. The push stops at `WITH_COLUMN`, `GROUP_BY`, `AGGREGATE`, and `JOIN`.

Correctness

Theorem 3. *For a row-independent operation op that does not produce any column in the set C ,*

$$\text{select}(C, op(T)) = op(\text{select}(C, T)).$$

Proof. `select`(C, T) retains columns C and leaves all rows and their values unchanged. op does not produce a column in C , so every column in C has the same value in $op(T)$ as in T . The row ordering is also unchanged. Therefore applying `select` before or after op produces the same table. $\square$

The push is blocked at `WITH_COLUMN` because the new column may be in C ; pushing past it would drop the column before it is created. It is blocked at `AGGREGATE` and `GROUP_BY` because those require the full pre-aggregation schema.

Example

Before:	SCAN -> FILTER(age > 30) -> SELECT(name, salary)
After:	SCAN -> SELECT(name, salary, age) -> FILTER(age > 30) -> SELECT(name, salary)

Performance benefit

Carrying fewer columns reduces memory bandwidth and the per-row cost of every subsequent operation.

6.4 Constant Folding

Transformation

A `BinaryExpr` whose both children are `LiteralExpr` nodes is replaced at plan-construction time by a single `LiteralExpr` holding the computed result. The transformation is applied recursively bottom-up.

Correctness

Theorem 4. For literals ℓ_1, ℓ_2 and binary operator $\oplus$,

$$\text{eval}(\text{BinaryExpr}(\ell_1, \ell_2, \oplus), T) = \text{eval}(\text{LiteralExpr}(\ell_1 \oplus \ell_2), T).$$

Proof. Both sides produce a vector of length $|T|$ where every element equals $\ell_1 \oplus \ell_2$, since neither operand depends on any row. Integer arithmetic preserves the integer type unless division is involved, where the result is promoted to float; both cases are handled explicitly. Division by zero is defined to produce 0.0 rather than undefined behaviour. $\square$

Example

```
Before:  with_column("x", lit(3) * lit(4) + col("salary"))
After:   with_column("x", lit(12) + col("salary"))
```

Performance benefit

Eliminates per-row computation for sub-expressions that can be fully resolved at plan-construction time.

6.5 Expression Simplification

Transformation

Algebraic identities are applied recursively to the expression tree:

Pattern	Simplified form
$x + 0, x - 0$	x
$x \times 1, x \div 1$	x
$x \times 0$	<code>lit(0)</code>
$\neg(\neg x)$	x

Correctness

Each rewrite is a standard algebraic identity. For the numeric rules, they hold over both integer and floating-point domains for all non-null values. Null propagation is unaffected: the simplified expression retains the same column reference, so any null in the column still produces a null result. For double-negation elimination, boolean negation is an involution ($\neg\neg b = b$ for $b \in \{\text{true}, \text{false}, \text{null}\}$), which can be verified by exhaustive case analysis on the three-valued logic.

Example

```
Before:  filter(~(~(col("active"))))
After:   filter(col("active"))

Before:  with_column("z", col("salary") * lit(1) + lit(0))
After:   with_column("z", col("salary"))
```

Performance benefit

Reduces the depth and node count of expression trees, cutting per-row evaluation cost and memory allocations during expression traversal.

7 Design Decisions

Immutable frames. Every operation returns a new `EagerDataFrame`; no frame is mutated in place. This makes chaining safe and avoids aliasing bugs.

Lazy copies. `LazyDataFrame::filter(...)`, etc., copy the frame object (which only copies the `ops_` vector, not the data) and push a new `Operation`. The data file is never opened until `collect()`.

Separate `GroupedDataFrame`. The `group_by` step returns a distinct type that only exposes `aggregate()`. This prevents calling aggregation on an ungrouped frame at compile time.

ExprPtr wrapper. `shared_ptr<Expr>` is wrapped in a thin `ExprPtr` struct that adds method chaining (`.abs()`, `.contains()`, etc.) and operator overloads. The wrapper is implicitly convertible to `shared_ptr<Expr>` so it composes with existing Arrow APIs without friction.

Conservative optimiser. All pushdown rules stop at operations that could change row count or schema. This errs on the side of correctness at the cost of occasionally missing an optimisation opportunity.